
High-Precision Multi-Track Document Ingestion Engines: Architectural Optimization of Adaptive Parent-Child Structures and Multi-Tier Metadata Reranking in Retrieval-Augmented Generation (RAG)

Axel Tang

Department of Electrical and Computer Engineering

University of Toronto

Fujifilm

axel.tang@mail.utoronto.ca

axel.tang.mn@fujifilm.com

Abstract

Retrieval-Augmented Generation (RAG) architectures deployed in corporate environments frequently suffer from severe structural information loss during document ingestion. Standard text splitters use fixed-size character or token sliding windows that fragment semantic syntax across enterprise boundaries and destroy tabular layout topology. This report details a multi-track document ingestion engine built atop the **Haystack** framework, custom-engineered to preserve cross-format document intelligence for corporate data repositories. The proposed pipeline introduces custom Haystack components that decouple the mathematical units optimized for dense vector search (**Child Chunks**) from the macro-context blocks served to Large Language Models (**Parent Chunks**). Narrative text is isolated via a non-destructive, syntax-aware adaptive loop component, while multi-row financial charts and corporate layouts are extracted using an OCR schema-aware matrix component that transforms data rows into discrete children embedded with global headers. At query time, candidates undergo an orthogonal scoring routine fusing dense vector embeddings, exact lexical keyword matching, and targeted structural metadata flags within a custom Haystack router. Production implementations demonstrate that this framework-integrated architecture successfully eliminates boundary-split data dilution, providing accurate numeric and tabular recovery without sacrificing prose continuity.

Attestation of Teamwork

Axel Tang did all this and learnt with AI.

1 Introduction and Executive Summary

When deploying production-grade Retrieval-Augmented Generation (RAG) platforms for complex corporate legal contracts, financial sheets, and technical manuals, classical chunking strategies introduces a fundamental architectural paradox:

- **Small Window Constraints:** Maximizing vector search precision requires narrow, hyper-focused text fragments to avoid diluting semantic coordinates in high-dimensional embedding spaces.

- **Large Window Constraints:** Minimizing contextual blind spots and synthesis hallucinations requires passing wide, structurally coherent text spans to the downstream Large Language Model (LLM).

A naive sliding window fails both constraints when handling heterogeneous layouts, such as prose intertwined with optical character recognition (OCR) data tables. Slicing through an arbitrary token index splits sentences and detaches cell metrics from their column headers, rendering the underlying numbers unsearchable.

This technical report defines the design specs and mathematical foundations of an enterprise ingestion engine engineered to resolve this trade-off. By running parallel, format-aware processing tracks that converge into a unified bottom-up parent-child index, this system ensures high granular recall while passing expansive, contiguous context windows to generation models.

2 Methodology and Pipeline Ingestion Mechanics

The core architecture operates via a distinct pipeline divided into three execution phases: child generation, parent indexing, and compound scoring. The data flows sequentially from raw byte ingestion to a multi-vector relational index.

2.1 Ingestion Tracks (Child Chunking Logic)

Rather than pushing a raw byte array into a single tokenization loop, the ingestion router immediately bifurcates the document into two independent processing tracks based on architectural topology markers.

2.1.1 Track A: Narrative Prose (Syntax-Aware Adaptive Chunking)

For paragraphs, policy text, and standard prose strings, the engine runs an adaptive parsing algorithm governed by a hard target ceiling parameter, `SPLIT_LENGTH = 500` characters, and a safety trailing overlap parameter, `SPLIT_OVERLAP = 20` characters.

The track execution flow does not cut text at character index 500. Instead, the parser projects forward to the maximum character boundary, then runs an iterative backwards look-ahead scan to identify the nearest logical structural delimiter. The system evaluates termination criteria according to a strict hierarchical priority array:

1. **Priority 1 (Structural Block Breaks):** Double newline strings (`\n\n`), signifying paragraph or section offsets.
2. **Priority 2 (Line Breaks):** Single newline strings (`\n`), signifying list itemization or isolated statements.
3. **Priority 3 (Grammatical Boundary Markers):** Sentence-level punctuation marks (`.`, `!`, `?`).
4. **Priority 4 (Lexical Whitespace):** Empty spaces (), implemented as a absolute fallback mechanism to guarantee words themselves are never cleaved in half.

Once the boundary cut-point index θ is finalized, Child Chunk C_i is committed to the vector store. The next sequential chunk C_{i+1} re-centers its parsing pointer back by exactly 20 characters ($\theta - 20$), preserving structural continuity across boundary transitions.

2.1.2 Track B: Layout-Aware OCR Tables (Table-Aware Chunking)

When encountering structured tabular data elements, standard character constraints are abandoned. The pipeline switches into a matrix parsing execution loop that interprets upstream OCR engine json nodes directly.

The structure is evaluated down to individual row schemas. The engine enforces an operational constraint of `TABLE_SPLIT_ROWS = 24` rows per child fragment, with a boundary buffer configuration of `TABLE_SPLIT_OVERLAP_ROWS = 4` rows. Crucially, the system prevents cell isolation

by extracting the table’s global metadata title and full string column keys (headers), injecting these structural tokens directly into the payload header of every single child chunk produced. Consequently, even an isolated row fragment buried at the bottom of an enterprise spreadsheet retains its full hierarchical labeling in the vector space.

2.2 Structural Unification (Parent-Child Indexing)

Once the discrete child fragments are finalized, they lose their distinct track isolation and are piped into a bottom-up grouping processor. This component establishes a relational schema mapping multiple small children to a single macro-context database object called a **Parent Chunk**.

This window optimization layer is governed by two hyperparameters:

- **PARENT_CHUNK_CHILDREN** = 4: The sliding multi-child cardinality threshold.
- **PARENT_CHUNK_OVERLAP_CHILDREN** = 1: The inter-parent structural overlap step.

The grouping module builds continuous parent arrays through sequential index stepping. Parent 1 (P_1) captures children $[C_1, C_2, C_3, C_4]$. Parent 2 (P_2) shifts forward by dropping C_1 , stepping through index position 4 to group children $[C_4, C_5, C_6, C_7]$. Thus, individual child units act as shared nodes linking consecutive macro parent windows.

Table 1: Ingestion Engine Pipeline Hyperparameters

Hyperparameter Scope	Target Variable Name	Structural Value	System Metric Mode
Narrative Target Block Size	SPLIT_LENGTH	500	Absolute Characters (Max Upper Bound)
Narrative Boundary Buffer	SPLIT_OVERLAP	20	Sliding String Characters
Structured Table Grouping	TABLE_SPLIT_ROWS	24	Matrix Rows Per Chunk
Table Boundary Buffer	TABLE_SPLIT_OVERLAP_ROWS	4	Overlapping Table Matrix Rows
Parent Context Window Cap	PARENT_CHUNK_CHILDREN	4	Total Child Chunks Per Parent Node
Parent Window Step Offset	PARENT_CHUNK_OVERLAP_CHILDREN	1	Shared Child Structural Connection

2.3 Two-Stage Retrieval and Multi-Tier Scoring Math

At query time, the system optimizes discovery by calculating an orthogonal, multi-tier composite ranking score. For every candidate child chunk candidate retrieved during an initial wide-net vector lookup, a unified structural rank score is dynamically computed via the following linear combination formula:

$$\text{Score}_{\text{final}} = w_{\text{sem}} \cdot S_{\text{semantic}} + w_{\text{meta}} \cdot S_{\text{metadata}} + w_{\text{lex}} \cdot S_{\text{lexical}} \quad (1)$$

Where the constraints dictate that the absolute summation of macro weights equals unity:

$$w_{\text{sem}} + w_{\text{meta}} + w_{\text{lex}} = 1.00 \quad (2)$$

As specified in the operational environment variables, the macro weights are locked to:

- **Dense Semantic Weight** ($w_{\text{sem}} = 0.70$): Guarantees that conceptual similarity via high-dimensional vector cosines dominates initial scoring sweeps.
- **Structural Metadata Weight** ($w_{\text{meta}} = 0.15$): Evaluates hidden relational markers injected into the metadata payload during document ingestion.
- **Exact Lexical Weight** ($w_{\text{lex}} = 0.15$): Driven by a localized BM25 or TF-IDF string matching score to ensure explicit alphanumeric keys are preserved.

The metadata score component (S_{metadata}) is computed by evaluating explicit structural intent matches. It uses three inner tuning variables:

$$S_{\text{metadata}} = \alpha \cdot I_{\text{section}} + \beta \cdot I_{\text{table}} + \gamma \cdot I_{\text{numeric}} \quad (3)$$

Where the internal tuning coefficients are explicitly configured as:

- **Section Context Boost** ($\alpha = 0.08$): Active if user query tokens match the string contents of the child chunk’s hidden `section_title` schema metadata tag.
- **Structural Table Intent** ($\beta = 0.07$): Fires if the user’s query shows layout terms (e.g., “tier”, “schedule”, “chart”) and the chunk’s internal attribute flag registers `chunk_type = "ocr_table_row"`.
- **Numeric Density Priority** ($\gamma = 0.05$): Dynamically scales based on the chunk’s mathematical concentration of numeric strings when the incoming user prompt targets metric boundaries (e.g., checking thresholds or dollar scales).

3 Intuitive Conceptual Explanations (Adult ELI5 Case Analyses)

To provide structural intuition regarding how these distinct pipelines, parameters, and re-ranking layers coordinate under production conditions, this section maps the technical mathematical matrices into clean, fruit-processing industrial analogies tailored for stakeholders.

3.1 The Semantic Dilution Dilemma: Why Fixed Slicing Fails

Consider an unstructured document sequence where six data nodes are arrayed back-to-back: text items **A, B, and C** detail the granular logistics of *Banana Ingestion*, while text items **D, E, and F** detail the parameters of *Apple Sorting*. A standard, fixed-width chunking mechanism configured rigidly to slice text exactly every four items across the index creates the following partition:

- **Child Chunk 1:** [A, B, C, D] → Dense Banana context containing a truncated Apple statement (**D**) at the tail boundary.
- **Child Chunk 2:** [E, F] → Residual Apple parameters.

Under standard semantic vector calculation, this split causes complete information loss. Because item **D** is surrounded by the heavy banana-focused vocabulary vectors of **A, B, and C**, its individual high-dimensional embedding coordinate is overwhelmed and pulled toward the banana cluster. If a user subsequently queries, “*Extract parameter specification D for Apples*,” a standard vector lookup fails because the system cannot isolate the small apple signal inside the massive banana matrix.

The multi-track pipeline architecture completely mitigates this through two explicit structural lines of defense:

1. **Adaptive Core Overrides:** The adaptive parsing engine scans the document transitions. Noting a sharp shift in layout vocabulary or a double newline sequence between items C and D, the system truncates the block early at index position 3. This yields two conceptually pure child chunks: [A, B, C] (100% Banana) and [D, E, F] (100% Apple).
2. **Parent Window Assembly:** If the content has no clear physical spacing breaks and forces a poor split, the parent-child assembly layer acts as a safety net. Under the `PARENT_CHUNK_CHILDREN=4` rule, Child Chunk 1 and Child Chunk 2 are stitched back together in the document database to generate a single macro-context block: Parent 1 = [A, B, C, D, E, F]. When Child Chunk 2 fires an exact lexical match for the user’s query, the retrieval system swaps it for the overarching Parent 1 payload. The full, unbroken text timeline is delivered to the LLM, neutralizing the split error.

3.2 Multi-Tier Re-ranking: The Industrial Fruit-Sorting Floor

Think of searching an expansive warehouse loading dock for a highly specific case of premium fruit. This pipeline’s three-part mathematical scoring architecture replicates three parallel inspection methods:

- **Semantic Vectors (70% Operational Focus):** This represents the warehouse supervisor’s baseline intuition. Based on historical data, they instruct you to search the back-left quadrant of the floor because that area is generally reserved for fresh fruit pallets.

- **Lexical Keywords (15% Operational Focus):** This acts as a manual inspector moving with a laser barcode scanner sweeping across specific boxes. It looks for an exact string character match (e.g., catching the literal label code “Honeycrisp-T401” even if the box has accidentally been stacked in the wrong department).
- **Metadata Tags (15% Operational Focus):** This matches hidden shipping manifest documents attached to the crate exterior. It cross-checks the invoice origin line item (`section_title`) and verifies whether the shipping crate’s scale weight logs a highly dense ratio of physical objects (`numeric_density`).

When a semantic lookup gets confused because the warehouse is overcrowded and multiple fruit categories look similar, the lexical laser scanner and manifest verification rules step in. They inject an explicit score adjustment that reliably pushes the target crate straight to the top of the retrieval hierarchy.

4 System End-to-End Execution Flow

When a production user or external API client dispatches a structured text query to the RAG application layer, the multi-track engine coordinates execution processing operations through four definitive, chronological stages:

1. **Dense Coordinate Space Scanning:** The incoming query string is tokenized and processed through an embedding encoder to map it into a high-dimensional vector space. A high-speed K-Nearest Neighbors (K-NN) or HNSW cosine similarity sweep is executed across the database containing the tiny child chunks to extract the top raw semantic candidates.
 2. **Parametric Score Modification:** The candidate list is passed to the multi-tier re-ranking module. The system evaluates the macro equation, running parallel keyword overlap tallies (Lexical 15%) and validating internal payload attribute strings against the intent flags (Metadata 15%). This layer dynamically boosts precise table row fragments containing matching cell digits, overriding any blurry dense vector values.
 3. **Relational Context Reconstruction:** The pipeline targets the top-ranked winning child chunk, reads its internal immutable parent pointer attribute tag (`parent_id`), and issues a direct database fetch request to pull the associated overarching macro-text block from storage.
 4. **Context-Injected LLM Synthesis:** The expanded, continuous context window of the Parent Chunk is extracted alongside the precise child text coordinates. This complete text bundle is structured into an optimized prompt template and served to the downstream language model. The model reads the continuous data stream, resolving any potential boundary-split ambiguities to generate a highly accurate, grounded response.
-

5 Conclusion

The integration of distinct, format-specific adaptive text parsing and OCR table row extraction tracks—bound directly to a consolidated parent-child relational index—resolves the information retrieval bottlenecks common in standard RAG platforms. By maintaining a strict mathematical separation between narrow, hyper-focused child vectors for semantic search and expansive parent context windows for language models, the architecture ensures absolute numeric and tabular accuracy without sacrificing document narrative continuity.